



כ"ב בשבט תשפ"ו

מס' שאלון - 462

9

בפברואר 2026

סמסטר 2026א

20441 / 4

מס' מועד 83

שאלון בחינת גמר

20441 - מבוא למדעי המחשב ושפת Java

משך בחינה: 3 שעות

בשאלון זה 15 עמודים

מבנה הבחינה:

- קראו בעיון את ההנחיות שלהלן:
- * עליכם לענות על חמש השאלות.
- * כל התכניות צריכות להיות מתועדות היטב.
- יש לכתוב תחילה בקצרה את האלגוריתם וכל הסבר נוסף הדרוש להבנת התכנית.
- יש לבחור בשמות משמעותיים למשתנים, לפונקציות ולקבועים שבתכנית.
- תכנית שלא תתועד כנדרש לעיל תקבל לכל היותר 85% מהניקוד.
- * יש להקפיד לכתוב את התכניות בצורה מבנית ויעילה.
- תכנית לא יעילה לא תקבל את מלוא הנקודות.
- * אם ברצונכם להשתמש בתשובתכם בשיטה או במחלקה הכתובה בחוברת השקפים, אין צורך שתעתיקו את השיטה או את המחלקה למחברת הבחינה.
- מספיק להפנות למקום הנכון,
- ובלבד שההפניה תהיה מדויקת (פרמטרים, מיקום וכו').
- * אין להשתמש במחלקות קיימות ב-Java, חוץ מאלו המפורטות בשאלות הבחינה.
- * יש לשמור על סדר; תכנית הכתובה בצורה בלתי מסודרת עלולה לגרוע מהציון.
- * בכתיבת התכניות יש להשתמש אך ורק במרכיבי השפה שנלמדו בקורס זה
- אין להשתמש במשתנים גלובליים!
- * אפשר לתעד בעברית. אין צורך בתיעוד API.
- * על שאלות 3-5 יש לענות אך ורק בשאלון ולא במחברת הבחינה!

חומר עזר:

חוברות השקפים 1-6, 7-12. אסור לכתוב כלום בתוך חוברות השקפים.
מותר לסמן עמודים בצבע או בדגלונים.
אין להכניס מחשב או מחשבון או מכשיר אלקטרוני מכל סוג שהוא.
אין להכניס חומר נוסף אחר מכל סוג.

החזירו

למשגיח את השאלון

וכל עזר אחר שקיבלתם בתוך מחברת התשובות

בהצלחה !!!



חלק א – עליכם לענות על כל השאלות בחלק זה במחברת הבחינה

שאלה 1 (25 נקודות)

נתון מערך דו-ממדי board המכיל תווים שהם אותיות ה-abc הקטנות בלבד. כמו כן נתונה מחרוזת תווים word המכילה מילה. אנחנו מעוניינים לבדוק אם ניתן להרכיב את המילה word במסלול במערך board בהתאם לכללים הבאים:

- המסלול מתחיל בתא שבו נמצאת האות הראשונה של המילה.
- בכל צעד ניתן לזוז לאחד מארבעת הכיוונים: למעלה, למטה, שמאלה או ימינה.
- לאורך כל המסלול מותר לבצע לכל היותר שינוי כיוון אחד בלבד.
- אין לבקר באותו תא פעמיים.

לדוגמא:

אם המערך הדו-ממדי board הוא זה:

	0	1	2	3	4
0	x	s	u	b	o
1	t	h	h	e	r
2	e	a	c	j	e
3	d	l	i	a	a
4	m	o	c	v	d
5	a	b	n	a	k

את המחרוזות "java" ו-"shalom" אפשר להרכיב לפי תנאי המסלולים. (המסלולים מסומנים במערך)

את המחרוזות "bread" ו-"teacher" אי אפשר להרכיב לפי תנאי המסלולים. במחרוזת "bread" (שמתחילה באות 'b' שנמצאת במקום board[0][3]) אי אפשר לעבור מ-'b' ל-'r' כי זה באלכסון, ובמחרוזת "teacher" (שמתחילה באות 't' שנמצאת במקום board[1][0]) יש יותר מפניה אחת.

כתבו שיטה סטטית בוליאנית רקורסיבית המקבלת כפרמטרים את המערך הדו-ממדי board מלא באותיות לטיניות קטנות בלבד, ומחרוזת תווים word, ומחזירה true אם אפשר להרכיב את word כמסלול במערך board לפי התנאים, ו- false אחרת.

חתימת השיטה היא:

```
public static boolean canFind(char[][] board, String word)
```

השיטה שתכתבו צריכה להיות רקורסיבית ללא שימוש בלולאות כלל. כך גם כל שיטות העזר שתכתבו (אם תכתבו) לא יכולות להכיל לולאות.

- אפשר להניח שהמערך מלא בנתונים חוקיים ואינו null. אין צורך לבדוק זאת.
- במהלך הפתרון אין להשתמש במערכי עזר או במבני נתונים נוספים.
- מותר לשנות את המערך בתוך השיטה, אבל הוא צריך לחזור למצב המקורי בסיומה.
- מתוך השיטות של המחלקה String מותר להשתמש אך ורק בשיטות length ו- charAt.
- אין צורך לדאוג ליעילות השיטה! אבל כמובן שצריך לשים לב לא לעשות קריאות רקורסיביות מיותרות!
- אל תשכחו לתעד את מה שכתבתם!

המשך הבחינה בעמוד הבא

שאלה 2 (25 נקודות)

סעיף א – (17 נקודות)

נתון מספר שלם לא שלילי n .

כתבו שיטה סטטית המקבלת כפרמטר מספר שלם לא שלילי n , ומחזירה מספר שלם המהווה את רצפת השורש הריבועי של n , כלומר המספר השלם הגדול ביותר x כך ש- $x^2 \leq n$.
(floor)

לדוגמא,

- אם $n = 16$ השיטה תחזיר את הערך 4 (כי $4^2 = 16$). כלומר, ל-16 יש שורש ריבועי שלם.
- אם $n = 15$ השיטה תחזיר את הערך 3 (כי $3^2 = 9$, $4^2 = 16$ אבל $n < 16$). כלומר, ל-15 אין שורש ריבועי שלם.

שימו לב,

עליכם לכתוב פתרון יעיל שאינו משתמש ב- `Math.sqrt` או בכל שיטה אחרת מהמחלקה `Math` (כלומר לממש את החישוב בעצמכם).

חתימת השיטה היא:

```
public static int myIntSqrt(int n)
```

סעיף ב – (8 נקודות)

נתון מערך חד ממדי `arr` ובו מספרים שלמים. לכל המספרים במערך מהתא הראשון (אינדקס 0) ועד התא באינדקס k , יש שורש ריבועי שלם, ולכל המספרים במערך מהתא באינדקס $k+1$ ועד לתא האחרון (אינדקס `arr.length-1`) אין שורש ריבועי שלם.

כתבו שיטה סטטית המקבלת כפרמטר מערך `arr` המלא במספרים שלמים, ומקיים את הכתוב לעיל. השיטה צריכה להחזיר את האינדקס k .

אתם יכולים להניח שהמערך אינו ריק, ויש בו מספרים שלמים לא שליליים. בהכרח כל המספרים שיש להם שורש ריבועי שלם נמצאים ברצף לפני כל המספרים שאין להם שורש ריבועי שלם. שימו לב שיתכן שלכל המספרים במערך אין שורש ריבועי שלם, ואז השיטה צריכה להחזיר -1 ויתכן שלכל המספרים יש שורש ריבועי שלם ואז השיטה צריכה להחזיר את האינדקס של התא האחרון במערך.

לדוגמא:

אם המערך arr מכיל את המספרים הבאים:

0	1	2	3	4	5	6
4	1	0	9	6	10	22

אזי השיטה תחזיר את הערך 3 כי בתאים 0 עד 3 יש מספרים שיש להם שורש ריבועי שלם ובתאים 4 עד 6 יש מספרים שאין להם שורש ריבועי שלם.

חתימת השיטה היא:

```
public static int findIndex(int[] arr)
```

שימו לב:

השיטות שתכתבו צריכות להיות יעילות ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

ציינו מהי סיבוכיות זמן הריצה ומהי סיבוכיות המקום של כל שיטה שכתבתם. הסבירו תשובתכם.

בפתרון סעיף ב עליכם להשתמש בשיטה שכתבתם בסעיף א, גם אם לא פתרתם אותה. גם כאן אסור להשתמש בשיטות של המחלקה Math.

אל תשכחו לתעד את מה שכתבתם!

המשך הבחינה בעמוד הבא

חלק ב - את התשובות לשאלות 3, 4 ו-5 יש לכתוב על גבי השאלון. לא נבדוק תשובות שייכתבו במקום אחר!

שאלה 3 (18 נקודות)

נניח שהמחלקה Node שלהלן מממשת צומת עץ בינרי שערכיו הם תווים (char).

```
public class ChNode
{
    private char _letter;
    private ChNode _leftSon, _rightSon;

    public ChNode (char letter) {
        _letter = letter;
        _leftSon = null;
        _rightSon = null;
    }

    public char getLetter() { return _letter; }
    public ChNode getLeftSon() { return _leftSon; }
    public ChNode getRightSon() { return _rightSon; }
}
```

המחלקה BinaryTree מאגדת בתוכה שיטות סטטיות לטיפול **בעץ בינרי**.

בין השיטות נתונות השיטות f, something ו- mystery הבאות:

```
public static int f(ChNode t)
{
    if (t == null)
        return 0;
    return 1 + Math.max(f(t.getLeftSon()),
                        f(t.getRightSon()));
}

public static String something (char ch, int number,
                                int ind, String temp)
{
    if (ind == number)
        return temp;
    return something (ch, number, ind+1, temp+ch);
}
```

```

public static String mystery(ChNode t)
{
    if (t == null)
        return "";

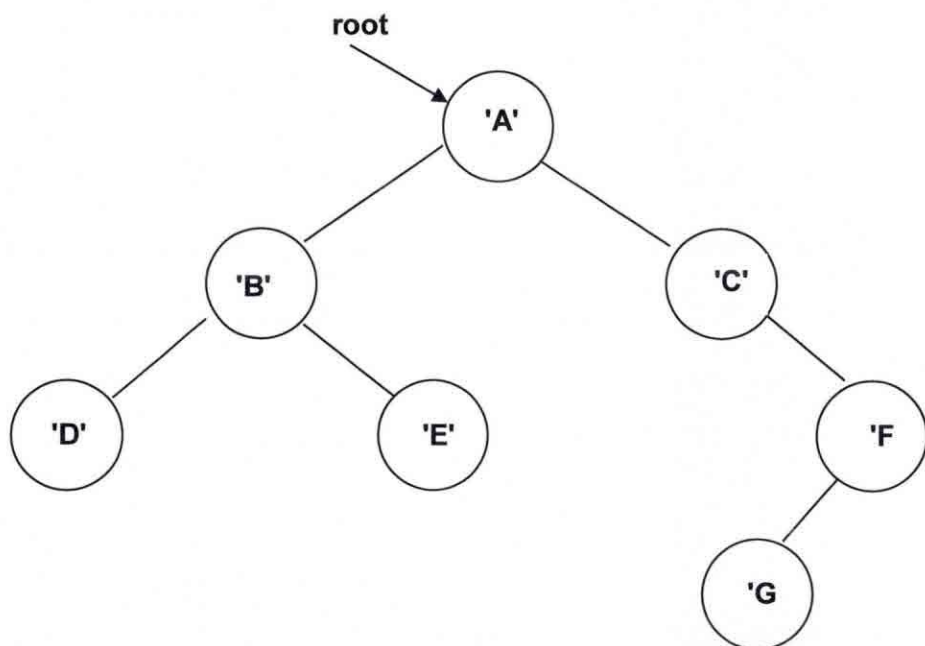
    int x = f(t.getLeftSon());
    int y = f(t.getRightSon());
    String local = something (t.getLetter(), x+y+1, 0, "");

    String s1 = mystery(t.getLeftSon());
    String s2 = mystery(t.getRightSon());

    return s1 + local + s2;
}

```

נתון העץ הבינרי הבא, ששורשו הוא root:



ענו על הסעיפים הבאים:

1. מה יודפס כתוצאה מהקריאה `?System.out.println(f(root));`

התשובה היא: (2 נקודות)



2. מה יודפס כתוצאה מהקריאה `?System.out.println(something('?', 5, 0, ""));`

התשובה היא: (3 נקודות)



3. מה יודפס כתוצאה מהקריאה `?System.out.println(mystery(root));`
התשובה היא: (5 נקודות)



4. לשיטה `mystery` הועבר כפרמטר שורש של עץ בינרי בשם `root`. המחרוזת שהוחזרה מהשיטה היא:

EDDCCCBBBBAAAAA

ציירו את העץ הבינרי ששורשו `root` שהועבר כפרמטר לשיטה.
אם לדעתכם אי אפשר לצייר עץ לפי המחרוזת לעיל, הסבירו מדוע אי אפשר.
אם לדעתכם יש יותר מעץ אחד שאפשר לצייר לפי המחרוזת, ציירו שני עצים כאלו.
התשובה היא: (4 נקודות)

_____	_____
_____	_____
_____	_____
_____	_____
_____	_____



5. בשיטה `mystery` שינו את פקודת ההחזרה:
`return s1 + local + s2;`
לפקודה הזו:
`return s1 + s2 + local;`
לשיטה `mystery` החדשה הועבר כפרמטר שורש של עץ בינרי בשם `root`. המחרוזת שהוחזרה מהשיטה היא:

EDDCCCBBBBAAAAA

ציירו את העץ הבינרי ששורשו `root` שהועבר כפרמטר לשיטה.
אם לדעתכם אי אפשר לצייר עץ לפי המחרוזת לעיל, הסבירו מדוע אי אפשר.
אם לדעתכם יש יותר מעץ אחד שאפשר לצייר לפי המחרוזת, ציירו שני עצים כאלו.
התשובה היא: (4 נקודות)

_____	_____
_____	_____
_____	_____
_____	_____
_____	_____

שאלה 4 (16 נקודות)

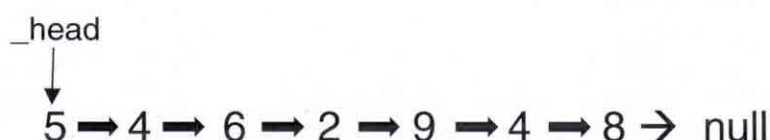
נתונה המחלקה `IntNode` הבאה, המייצגת איבר ברשימה מקושרת חד-סטרית המכילה מספרים שלמים:

```
public class IntNode
{
    private int _value;
    private IntNode _next;

    public IntNode(int val, IntNode n) {
        _value = val;
        _next = n;
    }

    public int getValue() { return _value; }
    public IntNode getNext() { return _next; }
}
```

נתונה הרשימה `list` הבאה:



לשם קיצור, נכתוב את הרשימה שלעיל כך, כאשר המספר משמאל הוא ראש הרשימה `_head`

{5, 4, 6, 2, 9, 4, 8}

נתונה רשימה מקושרת חד-סטרית, הממומשת בעזרת המחלקה `IntList` שמכילה את השיטות **הסטטיות** `secret1` ו-`secret2` הכתובות להלן. הניחו שיש שיטות נוספות במחלקה, כגון שיטה להוספת איבר לרשימה, שיטה למחיקת איבר מהרשימה, וכן השיטה `toString` המחזירה מחרוזת תווים המייצגת את איברי הרשימה. **כל אחת מהשיטות מקבלת כפרמטרים שתי רשימות חד-סטריות המכילות מספרים שלמים חיוביים בלבד.**

```
public class IntList
{
    private IntNode _head;

    public IntList() { _head = null; }
    public IntList(IntNode node) { _head = node; }
}
```

--- המשך המחלקה בעמוד הבא ---

שימו לב, בפקודות הראשונות של השיטה **secret1** יש קריאה לשיטה `mergeSort`. השיטה הזו מבצעת מיון-מיזוג על הרשימה עליה היא מופעלת, לפי הכתוב בחוברת השקפים ביחידה 11.5 בשקף 22.

להלן השיטה `secret1`:

```
public static int secret1 (IntList a1,IntList a2)
{
    a1.mergeSort();
    a2.mergeSort();
    int temp1=0, temp2=0;
    IntNode ptr1 = a1._head;
    IntNode ptr2 = a2._head;
    int curr = Integer.MAX_VALUE;
    while (ptr1 != null && ptr2 != null)
    {
        int max = Math.max(ptr1.getValue(),
                           ptr2.getValue());
        int min = Math.min(ptr1.getValue(),
                           ptr2.getValue());
        if (min == ptr1.getValue())
            ptr1 = ptr1.getNext();
        else
            ptr2 = ptr2.getNext();
        if (curr > (max - min))
        {
            curr = max - min;
            temp2 = max;
            temp1 = min;
        }
    }

    System.out.println(temp2 + ", " + temp1);
    return (temp2 - temp1);

} //end of secret1
```

```

public static int secret2 (IntList a1,IntList a2)
{
    IntNode ptr1 = a1._head;
    IntNode ptr2 = a2._head;
    int n1 = ptr1.getValue();
    int m1 = ptr1.getValue();
    int n2 = ptr2.getValue();
    int m2 = ptr2.getValue();

    while (ptr1 != null)
    {
        if (ptr1.getValue() < n1)
            n1 = ptr1.getValue();
        if (ptr1.getValue() > m1)
            m1 = ptr1.getValue();
        ptr1 = ptr1.getNext();
    }

    while (ptr2 != null)
    {
        if (ptr2.getValue() < n2)
            n2 = ptr2.getValue();
        if (ptr2.getValue() > m2)
            m2 = ptr2.getValue();
        ptr2 = ptr2.getNext();
    }

    if (m1-n2 > m2-n1)
    {
        System.out.println(m1 + ", " + n2);
        return m1-n2;
    }
    else
    {
        System.out.println(m2 + ", " + n1);
        return m2-n1;
    }
} //end of secret2
} //end of class IntList

```

ענו על הסעיפים הבאים:

סעיף א (8 נקודות – כל תת-סעיף 4 נקודות)

נתונות הרשימות list1 ו- list2 הבאות:

list1 $\rightarrow \{8, 1, 4, 15, 7, 4\}$ list2 $\rightarrow \{13, 17, 5, 12\}$

1. נקרא לשיטה secret1 כאשר הפרמטרים לשיטה הם הרשימות list1 ו- list2 שלעיל. איזה ערך יוחזר מהשיטה ומה יודפס כתוצאה מהקריאה `secret1(list1, list2)`?

התשובה היא:

הערך שיוחזר הוא: _____ יודפס: _____ 

2. נקרא לשיטה secret2 כאשר הפרמטרים לשיטה הם הרשימות list1 ו- list2 שלעיל. איזה ערך יוחזר מהשיטה ומה יודפס כתוצאה מהקריאה `secret2(list1, list2)`?

התשובה היא:

הערך שיוחזר הוא: _____ יודפס: _____ 

סעיף ב (8 נקודות – כל תת-סעיף 4 נקודות)

מה מבצעות השיטות secret1 ו- secret2 באופן כללי, כאשר הן מקבלות כפרמטרים שתי רשימות המכילות מספרים שלמים חיוביים בלבד?

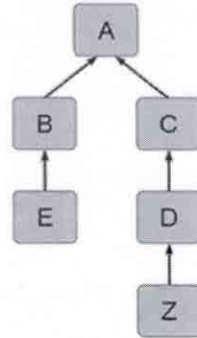
שימו לב, עליכם לתת תיאור ממצה של מה עושה כל שיטה באופן כללי, ולא תיאור של מה עושה כל שורה בשיטה, או איך היא מבצעת זאת. כלומר, מה המשמעות של הערך המוחזר מהשיטה, וכן מה מודפס כתוצאה מהפעלת השיטה. התייחסו למקרי קצה.

1. לגבי secret1 התשובה היא:

2. לגבי secret2 התשובה היא:

שאלה 5 (16 נקודות) -

בעץ הירושה להלן נתונות המחלקות A, B, C, D, E, Z.



אתם לא יכולים להניח שום דבר על המחלקות האלו (אם הן מחלקות רגילות, מופשטות או ממשקים). כך גם לא על התכונות שמופיעות בכל אחת מהן ולא על הבנאים ו/או השיטות הכתובות בהן.

ענו על שני הסעיפים הבאים:

סעיף א (6 נקודות – כל תת-סעיף נקודה אחת)

לכל אחת מהטענות הבאות, אם אתם חושבים שהטענה נכונה **תמיד** סמנו "נכון", אחרת סמנו "לא נכון". שימו לב שהטענה צריכה להיות **נכונה תמיד**, ולא רק במקרה מסוים. אין צורך לנמק.

1. משתנה מטיפוס מחלקה A יכול להצביע על אובייקט ממחלקה Z.

התשובה היא: נכון / לא נכון

2. המחלקה C לא יכולה להיות מופשטת (אבסטרקטית).

התשובה היא: נכון / לא נכון

3. אפשר ליצור מופע מהמחלקה B.

התשובה היא: נכון / לא נכון

4. המחלקה Z יכולה לגשת ישירה לכל התכונות שהוגדרו במחלקה C.

התשובה היא: נכון / לא נכון

5. המחלקה E יכולה לדרוס את כל השיטות שנמצאות במחלקה A.

התשובה היא: נכון / לא נכון

6. המרה (casting) מ-C ל-B לא עוברת קומפילציה.

התשובה היא: נכון / לא נכון



סעיף ב (10 נקודות – כל תת-סעיף 2 נקודות)

לפרויקט בו נמצאות כל המחלקות הנ"ל הוספנו מחלקה בשם Driver ובה הוספנו את השיטה
`public static void main(String[] args)`

ענו על השאלות הבאות:

1. אם נגדיר בכל אחת מהמחלקות שיטה ציבורית `public void doSomething()` מה יקרה בהפעלת הקוד הבא בשיטה main במחלקה Driver?

```
A obj = new D();  
obj.doSomething();
```

השיטה שתופעל תהיה השיטה שנמצאת במחלקה _____

2. מה תהיה התוצאה של הפעלת הקוד הבא בשיטה main במחלקה Driver?

```
B obj = new E();  
System.out.println(obj instanceof A);
```

סמנו את התשובה הנכונה:

- א. יודפס true .
- ב. יודפס false .
- ג. תהיה שגיאת קומפילציה .
- ד. תלוי אם A היא מחלקה מופשטת (abstract) .

3. במחלקה Z הוגדרה שיטה ייחודית בשם `specialZ()` . אם נכתוב את הקוד הבא בשיטה main במחלקה Driver תהיה שגיאת קומפילציה. מדוע?

```
D obj = new Z();  
obj.specialZ();
```

סמנו את התשובה הנכונה:

- א. כי לא ניתן להציב Z לתוך D.
- ב. כי הטיפוס המוצהר הוא D, ובמחלקה D לא מוגדרת השיטה `specialZ`.
- ג. כי השיטה חייבת להיות סטטית.
- ד. כי חסרה המרה ל-D.



4. איזו המרה, אם תיכתב בשיטה main במחלקה Driver, תגרום לשגיאת ריצה עקב המרה לא חוקית?

סמנו את התשובה הנכונה.

- א. `A obj = new C();` `D d = (D) obj;`
- ב. `A obj = new Z();` `D d = (D) obj;`
- ג. `C obj = new D();` `A a = (A) obj;`
- ד. `D obj = new Z();` `C c = (C) obj;`

5. בהנחה (בסעיף זה בלבד) שאף אחת מהמחלקות אינה מחלקה מופשטת (אבסטרקטית), איזו מהשורות הבאות תגרום לשגיאת קומפילציה, אם נכתוב אותה בשיטה main במחלקה Driver?

סמנו את התשובה הנכונה.

- א. `A obj = new Z();`
- ב. `B obj = new E();`
- ג. `C obj = new Z();`
- ד. `D obj = new A();`



ב ה צ ל ח ה !